

AI LEARNING

Mastering Artificial Intelligence Prompting, Python Infrastructure, and GitHub Model Deployment

BOOK NAME

AI Learning

CREATOR & WRITER

Muhammad Taqi

HUGGING FACE PROFILE[muhammad-taqi512](#)

CHAPTER 01

Introduction to Artificial Intelligence

Foundations & Prompting

Artificial Intelligence (AI) is a sophisticated computational system designed to process natural language inputs (prompts) and generate high-precision code, text, logic, and web applications according to exact user instructions.

1. Leading AI Model Ecosystems

Gemini

Google's ultra-fast multimodal AI model optimized for complex code syntax, long-context reasoning, and dynamic layout generation.

ChatGPT

OpenAI's flagship conversational LLM, excelling at logical step-by-step reasoning, natural language parsing, and creative coding.

DeepSeek

State-of-the-art open-weights model engine built for high-density mathematical derivations, structural logic, and efficient execution.

2. Theory: How AI Model Processing Works

An AI is essentially a program that executes task requests based on user prompts. The quality, specificity, and modern formatting of your prompt directly dictate the precision of the output generated.

1. User Prompt

Input Intent & Params

2. AI Inference Engine

Context & Pattern Match

3. Practical Output

Web / Code / Logic

3. Prompting Contrast Example: Portfolio Site

✓ HIGH-PRECISION PROMPT (RECOMMENDED)

"AIK VERY MODERN WEB BNAOO PORTFOLIO MY NAME IS MUHAMMAD TAQI AND MY WEB NAME IS NEXURA"

Result: AI perfectly understands the branding ("Nexura"), author identity ("Muhammad Taqi"), and generates a beautifully styled portfolio website.

✗ VAGUE PROMPT (FLAWED OUTPUT)

"AIK WEB BNA NAME NEXRA"

Result: AI cannot determine author identity, layout structure, or intended functionality, resulting in an incomplete or incorrect website.

4. Practical Prompt Exercises

Test these exact prompts in your AI interface to verify prompt performance:

1 Exercise 1: Chrome Dinosaur Game Clone

Prompt string: "AIK GAME BNAOO VERY VERY BEAUTIFULL NAME DIANO SAME LIKE DIANADOUR GAME OF GOOGLE CHROME"

Generates an interactive HTML5 canvas dinosaur jump game with collision detection and score tracking.

2 Exercise 2: Nexura Modern Web Portfolio

Prompt string: "AIK VERY MODERN WEB BNAOO PORTFOLIO MY NAME IS MUHAMMAD TAQI AND MY WEB NAME IS NEXURA"

Generates a futuristic personal portfolio page styled with modern web layouts and dynamic project cards.

CHAPTER 02

Python Programming for AI

Core Backend Architecture

Python is the universal language for AI software engineering. It connects user input, backend web services, JSON databases, and machine learning models into unified applications.

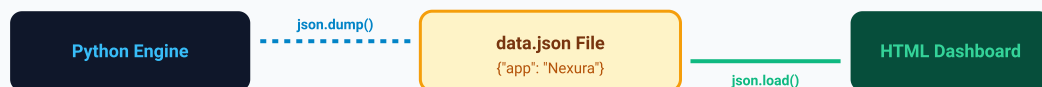
1. Handling User Input & Standard Output

Use Python's built-in `input()` and `print()` functions to capture user instructions and display AI results:

```
# Capturing user input prompt in Python user_prompt = input("Enter your AI prompt: ")
# Displaying output with formatted string (f-string) print(f"Executing Prompt for Developer:
Muhammad Taqi") print(f"Active User Command: {user_prompt}")
```

2. Structured Data Storage using data.json

AI backend applications use JSON (JavaScript Object Notation) files to persistently save prompt logs, model metrics, and system preferences.



```
import json # Writing configuration object to data.json dataset = {"project": "Nexura
AI", "author": "Muhammad Taqi", "status": "Active"} with open("data.json", "w") as file: jso
n.dump(dataset, file, indent=4) # Reading data from data.json with open("data.json", "r") as
file: loaded_data = json.load(file) print(loaded_data["project"])
```

3. Building Fast AI APIs with FastAPI

FastAPI allows developer applications to expose Python machine learning models over ultra-fast HTTP web endpoints.

```
from fastapi import FastAPI from pydantic import BaseModel app = FastAPI() class PromptModel(BaseModel): user_prompt: str author: str = "Muhammad Taqi" @app.post("/generate-ai") def process_ai(data: PromptModel): return { "status": "Success", "app_name": "Nexura AI Engine", "processed_prompt": data.user_prompt }
```

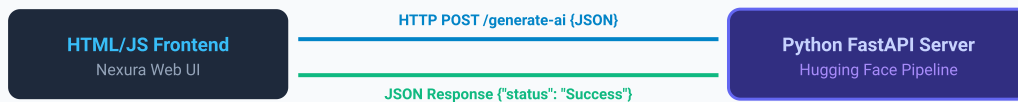
4. Loading Open-Source Hugging Face Models

Use Hugging Face's **transformers** library to load open-source models (such as those hosted on huggingface.co/muhammad-taqi512) directly inside Python code:

```
from transformers import pipeline # Initializing pipeline model from Hugging Face ai_generator = pipeline("text-generation", model="gpt2") # Executing inference prompt_input = "Nexura is a modern web framework built by Muhammad Taqi" ai_output = ai_generator(prompt_input, max_length=40, num_return_sequences=1) print(ai_output[0]['generated_text'])
```

5. Connecting Python AI Backend to HTML Frontend

Connect your Python FastAPI server with an interactive HTML web page using JavaScript asynchronous **fetch()** requests:



```
<!-- Frontend HTML + JavaScript Web Bridge --> <script> async function requestAICompletion() { const response = await fetch("http://127.0.0.1:8000/generate-ai", { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ user_prompt: "Build a game with modern graphics", author: "Muhammad Taqi" }) }); const result = await response.json(); console.log("AI Output:", result); } </script>
```

AI Model Developing & Deployment

Deploying custom AI models requires setting up automated cloud deployment pipelines. This chapter guides you through step-by-step repository deployment for the official project repository:

<https://github.com/NEXURA-M/LYRA>

Visual Deployment Pipeline



Step-by-Step Deployment Instructions

1 Step 1: Forking the Repository

Navigate to <https://github.com/NEXURA-M/LYRA> and click the **Fork** button in the top right header to create your copy under your developer organization.

2 Step 2: Configuring GitHub Pages Settings

Inside your forked repository, navigate to **Settings** → Select **Pages** in the left sidebar → Under *Build and deployment / Branch*, change the selector from **None** to **main** → Click **Save**.

3 Step 3: Granting Actions Read & Write Permissions

Go to **Settings** → Select **Actions** → Click **General** → Scroll down to *Workflow permissions* → Check **Allow Action to Read and Write** → Click **Save**.

4 Step 4: Triggering the GitHub Workflow

From the top navigation bar, click on the **Actions** tab → Select the active workflow under *All workflows* → Click **Run workflow** dropdown → Press the green **Run workflow** button to execute deployment.

AI LEARNING HANDBOOK COMPLETE

Author: Muhammad Taqi | Official HF Models: huggingface.co/muhammad-taqi512